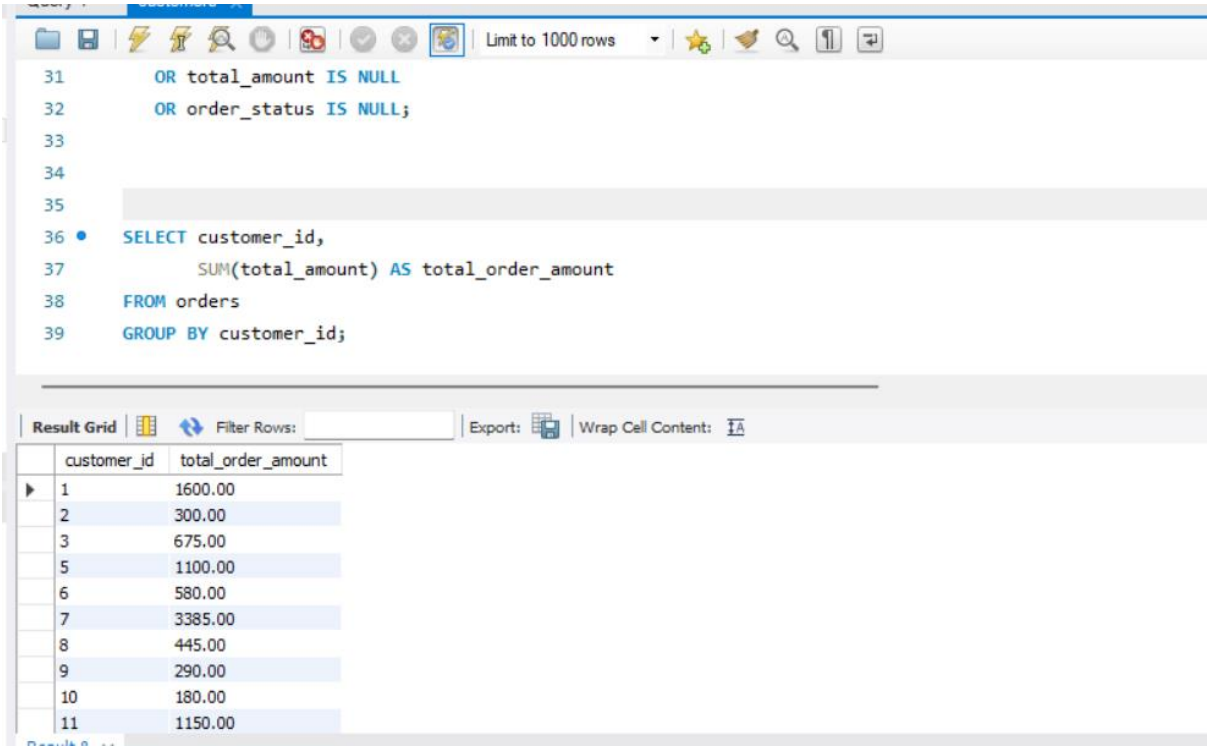
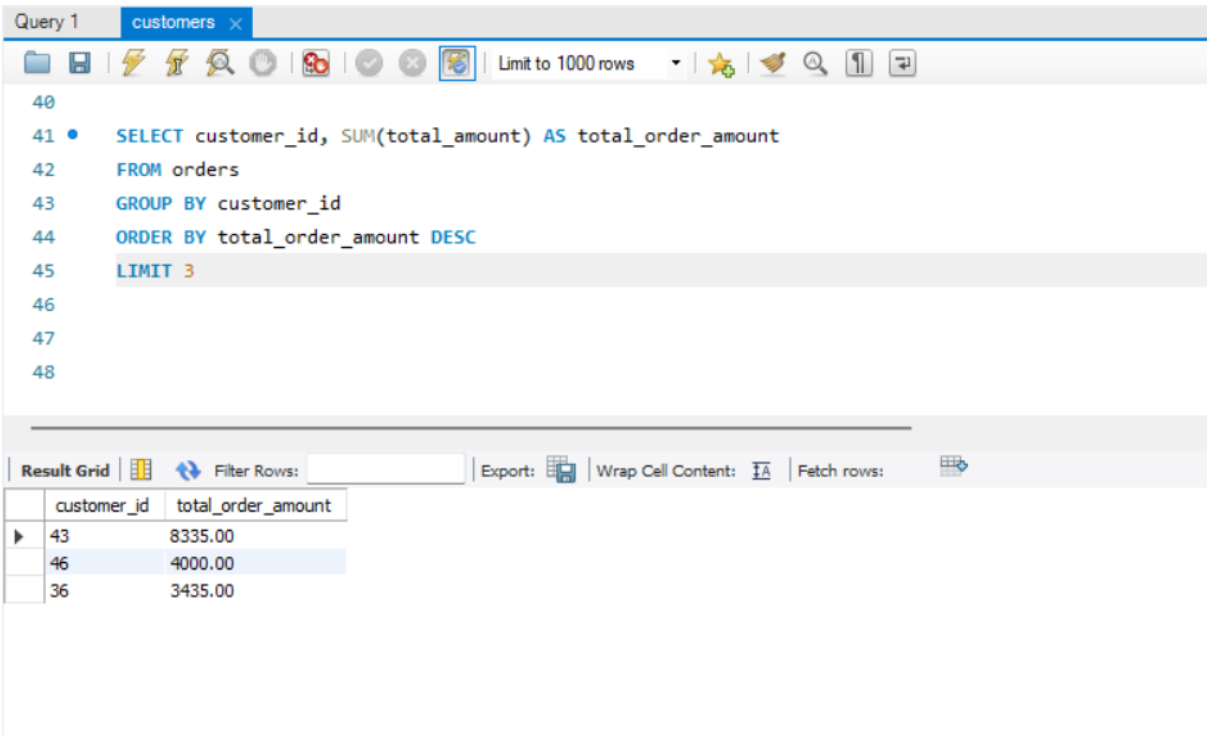


```
    OR total_amount IS NULL
    OR order_status IS NULL;


SELECT customer_id,
       SUM(total_amount) AS total_order_amount
FROM orders
GROUP BY customer_id;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

customer_id	total_order_amount
1	1600.00
2	300.00
3	675.00
5	1100.00
6	580.00
7	3385.00
8	445.00
9	290.00
10	180.00
11	1150.00



Query 1 | customers

Limit to 1000 rows

```

SELECT customer_id, SUM(total_amount) AS total_order_amount
FROM orders
GROUP BY customer_id
ORDER BY total_order_amount DESC
LIMIT 3


```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows:

customer_id	total_order_amount
43	8335.00
46	4000.00
36	3435.00

```sql
SELECT c.customer_id,
       c.first_name,
       c.last_name
FROM customers c
LEFT JOIN orders o
ON c.customer_id = o.customer_id
WHERE o.order_id IS NULL;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

customer_id	first_name	last_name
4	Liam	Johnson
27	Sofia	Young
32	James	Green
34	Lucas	Mitchell
35	Harper	Mitchell

```sql
SELECT c.city,
       SUM(o.total_amount) AS total_revenue
FROM customers c
JOIN orders o
ON c.customer_id = o.customer_id
GROUP BY c.city;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

city	total_revenue
Springfield	1600.00
Centerville	300.00
Greenville	675.00
Lakeside	1100.00
Oakland	580.00
Boise	3385.00
Des Moines	445.00
Albany	290.00
Portland	180.00
Miami	1150.00

```sql
SELECT customer_id,
       AVG(total_amount) AS avg_order_amount
FROM orders
GROUP BY customer_id;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

customer_id	avg_order_amount
1	1600.000000
2	150.000000
3	225.000000
5	1100.000000
6	290.000000
7	1692.500000
8	222.500000
9	145.000000
10	180.000000
11	575.000000

```sql
SELECT customer_id,
       COUNT(order_id) AS order_count
FROM orders
GROUP BY customer_id
HAVING COUNT(order_id) > 1;
```

Result Grid | Filter Rows: | Export: | Wrap Cell

customer_id	order_count
2	2
3	3
6	2
7	2
8	2
9	2
11	2
12	3
13	2
14	4

```sql
SELECT c.customer_id,
       CONCAT(c.first_name, ' ', c.last_name) AS customer_name,
       SUM(o.total_amount) AS total_spend
FROM customers c
JOIN orders o
ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.first_name, c.last_name;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

customer_id	customer_name	total_spend
1	John Smith	1600.00
2	Emma Jones	300.00
3	Olivia Brown	675.00
5	Noah Williams	1100.00
6	Alice Miller	580.00
7	Isabella Davis	3385.00
8	James Martinez	445.00
9	Sophia Garcia	290.00
10	Lucas Rodriguez	180.00
11	Mia Lopez	1150.00

1. Total order amount for each customer

```
from pyspark.sql.functions import sum as _sum
result1 = orders.groupBy("customer_id").agg(_sum("total_amount").alias("total_amount"))
result1.show()
```

```
+-----------+------------+
|customer_id|total_amount|
+-----------+------------+
|         31|      1450.0|
|         28|       180.0|
|         26|      3040.0|
|         44|      2200.0|
|         12|      1180.0|
|         22|       160.0|
|         47|      2280.0|
|          1|      1600.0|
|         13|       580.0|
|          6|       580.0|
|         16|       390.0|
|          3|       675.0|
|         40|      1820.0|
|         20|       220.0|
|         48|       800.0|
|          5|      1100.0|
|         19|      1180.0|
|         41|      2720.0|
|         15|       125.0|
|         43|      8335.0|
+-----------+------------+
only showing top 20 rows
```

2.Top 3 customers by total spend

```
top3 = (orders.groupBy("customer_id")
        .agg(_sum("total_amount").alias("total_amount"))
        .orderBy(col("total_amount").desc())
        .limit(3))
top3.show()
```

```
+-----------+------------+
|customer_id|total_amount|
+-----------+------------+
|         43|      8335.0|
|         46|      4000.0|
|         36|      3435.0|
+-----------+------------+
```

3.Customers with no orders

```
no_orders = (customers.join(orders, on="customer_id", how="left")
             .filter(col("order_id").isNull())
             .select("customer_id", "first_name", "last_name"))
no_orders.show()
```

```
+-----------+----------+---------+
|customer_id|first_name|last_name|
+-----------+----------+---------+
|          4|      Liam|  Johnson|
|         27|     Sofia|    Young|
|         32|     James|    Green|
|         34|     Lucas| Mitchell|
|         35|    Harper| Mitchell|
+-----------+----------+---------+
```

4. City-wise total revenue

```
city_revenue = (customers.join(orders, on="customer_id", how="inner")
                .groupBy("city")
                .agg(_sum("total_amount").alias("total_revenue")))
city_revenue.show()
```

```
+------------+-------------+
|        city|total_revenue|
+------------+-------------+
| Springfield|       1600.0|
|      Dallas|        800.0|
|     Oakland|        580.0|
| San Antonio|       2280.0|
|Indianapolis|       2810.0|
|   San Diego|       3500.0|
|   Nashville|       1180.0|
|     Detroit|       3510.0|
|    Portland|        180.0|
|      Albany|        290.0|
|       Boise|       3385.0|
|   St. Louis|       1445.0|
|    Columbus|         25.0|
|     Memphis|       1705.0|
|      El Paso|       2100.0|
|   Milwaukee|       3040.0|
|  Pittsburgh|        500.0|
|     Chicago|       8335.0|
| Centerville|        300.0|
|     Atlanta|        390.0|
+------------+-------------+
only showing top 20 rows
```

5.Average order amount per customer **bold text bold text**

```
from pyspark.sql.functions import avg
avg_per_customer = orders.groupBy("customer_id").agg(avg("total_amount").alias("avg_amount"))
avg_per_customer.show()
```

```
+-----------+------------------+
|customer_id|        avg_amount|
+-----------+------------------+
|         31|483.3333333333333|
|         28|             180.0|
|         26|             760.0|
|         44|            1100.0|
|         12|393.3333333333333|
|         22|             160.0|
|         47|             760.0|
|          1|            1600.0|
|         13|             290.0|
|          6|             290.0|
|         16|             195.0|
|          3|             225.0|
|         40|             910.0|
|         20|             220.0|
|         48|             800.0|
|          5|            1100.0|
|         19|             590.0|
|         41|906.6666666666666|
|         15|             125.0|
|         43|            1667.0|
+-----------+------------------+
only showing top 20 rows
```

6.Customers with more than one order

```
from pyspark.sql.functions import count
repeat_customers = (orders.groupBy("customer_id")
                    .agg(count("order_id").alias("order_count"))
                    .filter(col("order_count") > 1))
repeat_customers.show()
```

```
+-----------+-----------+
|customer_id|order_count|
+-----------+-----------+
|         31|          3|
|         26|          4|
|         44|          2|
|         12|          3|
|         47|          3|
|         13|          2|
|          6|          2|
|         16|          2|
|          3|          3|
|         40|          2|
|         19|          2|
|         41|          3|
|         43|          5|
|         37|          3|
|          9|          2|
|          8|          2|
|         39|          3|
|         23|          2|
|         49|          2|
|          7|          2|
+-----------+-----------+
only showing top 20 rows
```

7.Sort customers by total spend descending

```
sorted_spend = (customers.join(orders, on="customer_id", how="inner")
                .groupBy("customer_id", "first_name", "last_name")
                .agg(_sum("total_amount").alias("total_spend"))
                .orderBy(col("total_spend").desc()))
sorted_spend.show()
```

```
+-----------+----------+---------+-----------+
|customer_id|first_name|last_name|total_spend|
+-----------+----------+---------+-----------+
|         43|      Lily|   Turner|     8335.0|
|         46|   Michael|   Cooper|     4000.0|
|         36|     Mason| Campbell|     3435.0|
|          7|  Isabella|    Davis|     3385.0|
|         49|     Elena|     Gray|     3290.0|
|         26|   Matthew|    Allen|     3040.0|
|         25|    Harper|     Hall|     2730.0|
|         41|  Penelope|  Roberts|     2720.0|
|         50|     Henry| Phillips|     2450.0|
|         47|    Amelia|    James|     2280.0|
|         44|    Daniel|   Morris|     2200.0|
|         38|     Logan|   Carter|     2100.0|
|         45| Charlotte|     Wood|     1950.0|
|         40|  Benjamin|    Evans|     1820.0|
|         39|      Aria|    Davis|     1705.0|
|         14|     Ethan|   Taylor|     1635.0|
|          1|      John|    Smith|     1600.0|
|         31|     Chloe|    Adams|     1450.0|
|         24|    Daniel|   Walker|     1445.0|
|         17| Charlotte|   Harris|     1300.0|
+-----------+----------+---------+-----------+
only showing top 20 rows
```